

Jersey

De caras a la entrega final de la materia, comenzamos a ver que ventajas y requerimientos tiene una single-page application.

Destacamos que estas aplicaciones nos obligan a tener ya una API, que abre las puertas a que third-parties se integren con nuestro sistema y hace más fácil el día de mañana dar soporte a aplicaciones móviles. Además, nos permite mejorar tiempos de respuesta en la webapp, al minimizarse el contenido a ser transferido, pero tiene la contra de que la descarga inicial de la aplicación tiende a ser más grande.

El como cuidar el peso de la descarga inicial, el impacto de cache-pruning ante cambios y otros temas, se verá como parte del aprendizaje de AngularJS.

En clase charlamos sobre que implica una API REST, discutimos como modelar adecuadamente los paths a los distintos recursos, que status codes son esperables en general ante cada acción, y como resolver algunos problemas recurrentes como paginación. También hablamos sobre algunas mejores prácticas como ser la definición (y correcta documentación) de nuestros propios content-types, y como esto aplica a facilitar cuestiones como el versionado de la API.

Finalmente, vimos que si la API es realmente RESTful, tiene que ser stateless. Por tanto, depender de sesiones / cookies para autenticación no es una alternativa válida. Normalmente para esto se hace una de dos cosas:

1. se usa un token (enviado por header idealmente) sobre https (en el servidor de la materia no contamos con https, pero no es problema para la evaluación). Esquemas de este tipo son OAuth y JWT (recomendado para nuestro caso de uso)
2. se usa HMAC o un mecanismo similar para firmar el request con un token descartable (https deseable)

A modo de referencia, se cita la API de Github como un excelente ejemplo a seguir en como resuelve todos y cada uno de estos problemas.

Para implementar la API vamos a usar Jersey. Jersey es una implementación, por la gente de JBoss, al standard JAX-RS 2.0 (JSR 339). La versión que usaremos es la 3.0-SNAPSHOT, por ser esta la única compatible con Spring 4. Si bien es un snapshot, está lo suficientemente avanzada y estable como para ser utilizada en este TP. De esta forma, Jersey reemplazará a spring webmv para la implementación de controllers, pero seguiremos usando Spring para todo lo demás, desde security hasta persistencia.

Comenzamos entonces agregando la dependencia en el pom padre:

```
<!-- JERSEY -->
<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-servlet-core</artifactId>
  <version>${jersey.version}</version>
</dependency>
<dependency>
  <groupId>org.glassfish.jersey.ext</groupId>
  <artifactId>jersey-spring5</artifactId>
  <version>${jersey.version}</version>
</dependency>
</dependency>
```

```

<groupId>org.glassfish.jersey.core</groupId>
<artifactId>jersey-server</artifactId>
<version>${jersey.version}</version>
<exclusions>
  <exclusion>
    <groupId>javax.validation</groupId>
    <artifactId>validation-api</artifactId>
  </exclusion>
</exclusions>
</dependency>
<!-- JERSEY JSON -->
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-moxy</artifactId>
  <version>${jersey.version}</version>
</dependency>
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-jaxb</artifactId>
  <version>${jersey.version}</version>
</dependency>

```

y definiendo la versión correspondiente:

```
<jersey.version>2.43</jersey.version>
```

Y en el pom de webapp agrgeamos la dependencia:

```

<dependency>
  <groupId>org.glassfish.jersey.containers</groupId>
  <artifactId>jersey-container-servlet-core</artifactId>
</dependency>
<dependency>
  <groupId>org.glassfish.jersey.ext</groupId>
  <artifactId>jersey-spring5</artifactId>
</dependency>
<dependency>
  <groupId>org.glassfish.jersey.core</groupId>
  <artifactId>jersey-server</artifactId>
</dependency>
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-moxy</artifactId>
</dependency>
<dependency>
  <groupId>org.glassfish.jersey.media</groupId>
  <artifactId>jersey-media-jaxb</artifactId>
</dependency>

```

Corremos `mvn eclipse:eclipse` y actualizamos el proyecto.

Editamos el `web.xml`, y eliminamos el `DispatcherServlet` y su mapeo, para agregar en su lugar a Jersey.

```

<filter>
  <filter-name>jersey</filter-name>
  <filter-class>org.glassfish.jersey.servlet.ServletContainer</filter-class>
  <init-param>
    <param-name>jersey.config.server.provider.packages</param-name>
    <param-value>ar.edu.itba.paw.webapp.controller</param-value>
  </init-param>
  <!-- pass to next filter if Jersey/App returns 404 -->
  <init-param>
    <param-name>jersey.config.servlet.filter.forwardOn404</param-name>
    <param-value>true</param-value>
  </init-param>
</filter>

<filter-mapping>
  <url-pattern>*</url-pattern>
  <filter-name>jersey</filter-name>
</filter-mapping>

```

Es importante que el `filter-mapping` este último, por detrás del de `OpenEntityManagerInView` y el de `Spring Security` para procurar que estos apliquen primero.

Finalmente, definimos nuestro recurso REST en `ar.edu.itba.paw.webapp.controllers.UserController`

```

@Path("users")
@Component
public class UserController {

    @Autowired
    private UserService us;

    @Context
    private UriInfo uriInfo;

    @GET
    @Produces(value = { MediaType.APPLICATION_JSON, })
    public Response listUsers() {
        final List<User> allUsers = us.getAll();

        return Response.ok(new GenericEntity<List<User>>(allUsers) {}).build();
    }

    @POST
    @Produces(value = { MediaType.APPLICATION_JSON, })
    public Response createUser(final UserDTO userDto) {
        final User user = us.register(userDto.getUsername(), userDto.getPassword());

        final URI uri = uriInfo.getAbsolutePathBuilder()
            .path(String.valueOf(user.getId())).build();
        return Response.created(uri).build();
    }
}

```

```

    }

    @GET
    @Path("/{id}")
    @Produces(value = { MediaType.APPLICATION_JSON, })
    public Response getById(@PathParam("id") final long id) {
        final User user = us.getById(id);

        if (user != null) {
            return Response.ok(new UserDTO(user)).build();
        } else {
            return Response.status(Status.NOT_FOUND).build();
        }
    }

    @DELETE
    @Path("/{id}")
    @Produces(value = { MediaType.APPLICATION_JSON, })
    public Response deleteById(@PathParam("id") final long id) {
        us.deleteById(id);

        return Response.noContent().build();
    }
}

```